# Aurora

Complete project documentation

**Version** 1.6.1
**What it is** a free, open-source music player for the songs on your own device, with an AI DJ
**Platforms** Web, Android, Windows
**Source** github.com/Kailu687/Aurora
**Website** kailu687.github.io/Aurora
**Licence** MIT
**Generated** 2026-10-19

# Contents


## 1. Project guide

1. 1. What Aurora is
2. 2. Feature list (what the user sees)
3. 3. Repository map
4. 4. The web app, file by file (web/js/)
5. 5. How songs get from your folder to the player
6. 6. The AI DJ
7. 7. Platform differences
8. 8. The Android app in detail
9. 9. Updates (over the air)
10. 10. Building and releasing (developer guide)
11. 11. Troubleshooting
12. 12. Glossary

## 2. Releasing and publishing updates

1. A. One-time setup (about 10 minutes)
2. B. Publishing a new version (every time)
3. C. Attaching the files to a GitHub Release (so the Windows button works)
4. D. Which changes reach users how
5. E. Checking that updates work
6. F. Troubleshooting releases


Part 1 explains what Aurora is, how it is built, where every file lives, how songs and updates flow, and how to fix common problems. Part 2 is the step-by-step checklist for publishing a new version.

PART 1

# Aurora: complete project guide

This document explains the whole project: what it is, how it is put together, where every piece lives, how songs get from your folder to your ears, how updates reach the apps, and how to build and release it. You do not need to know the code to read the first sections. The later sections are for developers.

## 1. What Aurora is

Aurora is a music player for **the songs already on your own device**. It looks like a mix of Spotify and Apple Music (dark, frosted-glass panels, colours that follow the album art) and has an **AI DJ** that builds a set from your own library based on a mood you pick.

It exists in three forms that share **one code base**:

| Form | What it is | Where users get it |
|---|---|---|
| **Web app** | A website that also installs like an app (PWA) | `https://kailashamarnath.me/Aurora/app/` |
| **Android app** | The same web app inside a thin Android shell, plus native pieces for the notification player, folder access, and text-to-speech | `Aurora.apk` on `https://kailashamarnath.me/Aurora/downloads/` |
| **Windows app** | The same web app inside Electron, plus a native file server and folder scanner | `Aurora-Setup-x.y.z.exe` (GitHub Releases, linked from the Downloads page) |

The project also has a **marketing website** at `https://kailashamarnath.me/Aurora/` (features, open-source and security information, and a Downloads page with the release notes of every version). It lives in `tools/site/`.

Key promises:

- **Songs are never copied.** Aurora remembers *where* your folder is and reads the files from there (Android: a permission to the folder; Windows: the folder path; Chrome/Edge: a folder handle). Only small things are stored by Aurora: song titles, cover thumbnails, play counts, playlists, settings.
- **Everything works offline** after the first load. The only network use is (a) update checks and (b) looking up missing album covers, both described below, and (c) the optional AI-written DJ lines if you enter an API key.
- **Updates are over the air.** When a new version is published, installed apps notice and ask before updating.

## 2. Feature list (what the user sees)

- Library from a linked music folder: MP3, M4A, MP4, WAV, FLAC, OGG, OPUS (what plays depends on the device's decoders)
- Home with Daily Mixes, On Repeat, Rediscover, decade mixes, Chill and Power mixes, recently played/added
- Search; Albums, Artists, Genres, Songs; Liked Songs; playlists; recently played history
- Queue with drag-to-reorder, Play next, Add to queue, shuffle, repeat, autoplay of similar songs, song radio
- **AI DJ**: 12 vibes, optional genre filter, 3 DJ personalities, spoken intros that duck the music, live requests ("play some jazz", "more energy"), and nudges (Hype it up / Chill it down / More like this / Change it up)
- Crossfade, volume normalisation, speed, sleep timer
- Equalizer and visualizer (web and Windows; Android uses plain audio output, see section 7)
- Synced lyrics ( `.lrc` files next to songs, or lyrics stored in the tags) and plain lyrics
- Covers: embedded picture, or `cover.jpg` / `folder.jpg` in the folder, or looked up online once and saved
- Notification-bar / lock-screen / Samsung media-panel player on Android, media keys on Windows
- Resume where you left off (queue, song, position)
- Stats page ("listening personality"), backup/restore of playlists and settings
- Light and dark themes, accent colours, adjustable glass blur
- Over-the-air updates, with a prompt before installing
- Settings: **Reset to defaults**, **AI DJ on/off** (hides the DJ everywhere)

## 3. Repository map

```
Aurora/
├─ web/                    THE APP. Everything users see. Shared by web, Android and Windows.
│  ├─ index.html           Page skeleton: sidebar, player bar, now-playing screen, containers
│  ├─ css/style.css        All styling (glassmorphism, layouts, phone layout, themes)
│  ├─ js/                  The program, one file per concern (section 4)
│  ├─ sw.js                Service worker: offline cache + holds downloaded updates
│  ├─ manifest.webmanifest PWA manifest (name, icons)
│  ├─ config.json          { "updateUrl": ".../Aurora/app/" }  where updates come from
│  └─ icons/               App icons (PNG)
│
├─ android/                Android project (Gradle). A thin native shell around web/
│  └─ app/src/main/
│     ├─ java/app/aurora/player/
│     │  ├─ MainActivity.java        Starts the web view, folder picker, back button
│     │  ├─ AuroraBridge.java        The native functions the web app can call (window.AuroraAndroid)
│     │  ├─ PlayerService.java       The media notification + background playback keep-alive
│     │  └─ MediaWebViewClient.java  Serves your songs to the player straight from the chosen folder
│     ├─ AndroidManifest.xml         Permissions, the service, app name and icon
│     └─ res/                        Icons, colours, splash
│
├─ electron/               Windows (and desktop) shell
│  ├─ main.js              Window, app:// file server, folder scan, OTA
│  └─ preload.js           The small, safe bridge between Electron and the web app (window.desktop)
│
├─ tools/                  Developer scripts
│  ├─ serve.js             Local web server for development (npm run web)
│  ├─ publish-web.js       Version bump (npm run bump) + website builder (npm run site)
│  ├─ hash-releases.js     Size + SHA-256 of the APK/installer for the Downloads page
│  ├─ build-android.ps1    Builds the signed APK (npm run android)
│  ├─ build-all.ps1        APK + Windows installer + checksums (npm run release)
│  ├─ make-icons.js        Draws the PNG icons without any image library
│  ├─ make-test-music.js   Creates small fake songs for testing
│  └─ site/                The marketing website (home page, downloads page, css, screenshots)
│
├─ downloads/              Committed release files served by the website: Aurora.apk, hashes.json
├─ docs/changelog.json     Release notes of every version (feeds the Downloads page)
├─ .github/workflows/      GitHub Actions: publishes the website on every push (pages.yml)
├─ docs/                   This documentation
├─ build/icon.png          Icon used by the Windows installer
├─ capacitor.config.json   Tells Capacitor (the Android wrapper) to use web/ as the app
└─ package.json            Scripts, versions, Windows installer settings
```

Things that are **not** in git on purpose: `node_modules/`, build output, and the **Android signing key** (`android/keystore.properties` and the `.jks` file; see section 10).

## 4. The web app, file by file (`web/js/`)

The scripts load in this order (see `index.html`) and share one global object `A`:

| File | Responsibility |
|---|---|
| `version.js` | `A.VERSION = {name, build, notes}`. Written by `npm run bump`. The **build** number is what update checks compare. |
| `util.js` | Small helpers: DOM shortcuts, time formatting, hashing, the icon set, toasts, `A.store` (settings in `localStorage`, keys start with `aurora:`) |
| `db.js` | Tiny IndexedDB wrapper. Database `aurora-db` with stores `tracks`, `art`, `files`, `roots`, `kv` |
| `meta.js` | Reads song information: ID3v1/v2 and MP4 tags, cover pictures, duration, lyrics; makes cover thumbnails; analyses audio for the DJ (energy, tempo); parses `.lrc` |
| `engine.js` | The audio engine: two "decks" for crossfading, equalizer, ducking (lowering music while the DJ talks), analyser for the visualizer. Has a **direct mode** (plain `<audio>`) used on Android |
| `library.js` | The library: scanning folders on each platform, adding songs instantly, reading tags in the background, cover lookup, albums/artists/genres, playlists, lyrics, smart mixes, search |
| `player.js` | Playback logic: queue, shuffle, repeat, history, crossfade timing, "resume where you left off", sleep timer, lock-screen metadata |
| `dj.js` | The AI DJ: vibes, song scoring, set building, similar songs, the background analysis queue, speech (browser voices or phone text-to-speech), line writing |
| `updater.js` | Over-the-air updates (section 9) |
| `ui.js` | Reusable UI: colour extraction from art, modals, context menus, row and card builders, "edit song info" |
| `views.js` | Every page (Home, Search, Library, Album, Artist, DJ, Stats, Settings...) and the router |
| `shell.js` | The frame around the pages: player bar, Now Playing screen, queue/lyrics panels, button actions, importing, keyboard shortcuts, back-button logic, start-up |
| `native.js` | Android only: connects the web app to the native side (notification player, folder picker, covers over HTTPS, text-to-speech) |

## Data model (what is stored)

A **track** record (IndexedDB `tracks`) holds: id, title, artist, album, genre, year, duration, `artKey` (which cover), `kind` (`handle` = browser folder handle, `path` = Windows file path, `saf` = Android document, `blob` = copied file for browsers that cannot link folders), the location (`handle` / `path`), plays, last played, liked, vibe analysis, `meta` (0 = tags not read yet, 1 = done), `coverTried`.

- `art` store: small JPEG cover thumbnails (about 400 px), keyed by album.
- `files` store: only used by the "copy" fallback for browsers that cannot link a folder (iPhone Safari, Firefox).
- `roots` store: the linked folders.
- Playlists, history, resume position and settings are in `localStorage` (`aurora:*`).

# 5. How songs get from your folder to the player

**Adding a folder** (all platforms follow the same two phases):

1. *Instant phase*. The platform lists the audio files in the folder. Aurora creates one record per new file using the file name ("Artist - Title") so the library appears immediately.
2. *Background phase* ( `L.enrich` in `library.js` ). Three at a time, Aurora reads each file's tags (title, artist, album, genre, year, duration, embedded cover, lyrics), without blocking anything. A small chip shows progress.
3. *Cover phase* ( `L.fetchCovers` ). Songs still without a picture get one: first a `cover.jpg` etc. from the same folder, otherwise a one-time lookup on Apple's public catalog (artist/album names only; strict title+artist matching so wrong covers are avoided). The result is saved, so it is never fetched twice.

**Playing a song:** the engine needs a URL for the audio element.

| Platform | Where the bytes come from |
|---|---|
| Android | `https://localhost/_media/<encoded document id>` . `MediaWebViewClient` intercepts that request and streams the file from the chosen folder with HTTP Range support (so seeking works) |
| Windows | `app://local/__lib/<encoded path>` . `electron/main.js` streams the file from disk with Range support |
| Chrome / Edge | The file handle is turned into a temporary `blob:` URL (still not copied; it points at the original file) |
| Other browsers | The copied blob from IndexedDB |

**Reading a song's tags** needs only the first few hundred KB, fetched with Range requests, so a 600-song library is read quickly and with little memory.

# 6. The AI DJ

*Not a cloud AI by default.* It is a local "taste engine":

1. **Vibe analysis** ( `meta.js: A.analyze` ): in the background, for each song, a short slice is decoded and measured for loudness, tempo and brightness, giving an *energy* score. Stored on the track. Runs only when the DJ is on.
2. **Scoring** ( `dj.js: A.vibeScore` ): each vibe (Chill, Hype, Focus, Late Night, Workout, Party, Feel Good, In My Feelings, Romance, Throwback, Road Trip, Wind Down) defines target energy, tempo, genre keywords and words. A song's score combines genre/keyword match, energy fit, tempo fit and era.
3. **Set building** ( `dj.js: dj._pick` ): picks the next song among the best few, with randomness, avoiding the same artist twice in a row, following a gentle energy wave, preferring smooth tempo/energy transitions. User nudges shift the target; "More like this" biases towards similar songs.
4. **Talking**: at each song start the DJ writes a line from templates (style: Smooth / Hype / Zen) using real facts about the song (year, album, how often you played it, tempo, time of day) and speaks it, lowering

the music while talking. Voices come from the browser/OS, or from the phone's text-to-speech on Android.

5. *Optional*: if you paste an Anthropic API key in Settings, the line is written by Claude instead (only song and artist names are sent). The key stays on the device.

Switch the whole feature off in **Settings, AI DJ, Enable AI DJ**.

## 7. Platform differences

| | Web (Chrome/Edge) | Web (other) | Android app | Windows app |
|---|---|---|---|---|
| Link a folder (no copying) | Yes | No (copies files) | **Yes** | **Yes** |
| Notification / lock-screen player | Browser media controls | Limited | **Yes (native)** | Media keys |
| Equalizer + visualizer | Yes | Yes | No (plain audio output) | Yes |
| Background playback | While the tab lives | Limited | **Yes** (foreground service) | Yes |
| AI DJ voice | Browser voices | Browser voices | Phone text-to-speech | System voices |
| Over-the-air updates | Yes (service worker) | Yes | Yes | Yes |

**Why no equalizer on Android:** Android's web view can output silence when audio goes through the Web Audio effects chain, so the Android app uses plain audio output ( `engine.js` "direct mode"). Volume, crossfade, normalisation, ducking, speed and the sleep timer still work.

**Closing the app:** on Android, swiping Aurora away from the app switcher stops the music and the service and ends the process ( `PlayerService.onTaskRemoved` ). The Back button steps back through pages and only sends the app to the background from the home screen.

## 8. The Android app in detail

- **Capacitor** ( `capacitor.config.json` ) packages `web/` into an app that opens it inside the Android web view at `https://localhost` . `npx cap sync android` copies `web/` into `android/app/src/main/assets/public/` (that folder is generated and not committed).
- `AuroraBridge` is exposed to the page as `window.AuroraAndroid` . Functions: `pickFolder` , `scan` , `hasAccess` (folder access); `update` , `stop` (notification player); `speak` , `stopSpeak` (text-to-speech); `httpGet` (cover lookup, only Apple hosts allowed); `status` , `requestPermission` , `openSettings` (notifications). Native to web messages go through `A.nativeEvent(...)` and `A.nativeCmd(...)` in `native.js` .

- **Folder access** uses Android's Storage Access Framework (`ACTION_OPEN_DOCUMENT_TREE`). The permission is saved persistently, so the folder stays linked across restarts and app updates.
- `PlayerService` shows a media-style notification with Previous/Play-Pause/Next, cover art and a seek bar. This is the standard Android media session, which is also what Samsung's lock-screen and quick-panel players show. Android 13+ needs the notifications permission; Aurora asks once after you start your first song.
- `MediaWebViewClient` answers requests to `/_media/...` with file data from the folder (with Range support).

## 9. Updates (over the air)

```
  you change code ─► npm run bump ─► commit + push ─► GitHub Actions builds the website ─► GitHub
Pages
                                                                                          │
   installed apps (web, Android, Windows) ◄── fetch update-manifest.json ◄────────────────┘
        │ newer "build" number? ─► ask the user ─► download listed files ─► restart on the new version
```

- `update-manifest.json` is generated by `tools/publish-web.js` and lists the app's files with sizes plus the build number and release notes.
- **Where the app looks:** `web/config.json` (`updateUrl`, currently `https://kailashamarnath.me/Aurora/app/`). If the app itself is served from that website, its own address is used.
- **When:** a few seconds after launch, every 6 hours while open, and when you press **Check for updates** in Settings.
- **How it applies:**
  - Browser / Android: the files are saved in the browser cache `aurora-live`; the service worker `sw.js` serves them before anything else. A newer APK automatically discards an older downloaded update.
  - Windows: `electron/main.js` downloads to the user-data folder `web-live/` and serves it instead of the bundled files.
- **What updates cannot change:** native code. Changes to the Android Java files or `electron/main.js` need a new APK or installer. Everything under `web/` (the large majority of features) updates over the air.

## 10. Building and releasing (developer guide)

Requirements: Node 20+, JDK 17+ and the Android SDK (Android Studio provides both), Windows for the installer.

```
npm install
npm run web          # http://localhost:5173, for development
npm start            # run the Windows/desktop app from source
npm run bump -- --notes "First change|Second change"   # new version number everywhere + changelog
entry
npm run android      # signed APK  -> downloads/Aurora.apk
npm run windows      # installer   -> dist/Aurora-Setup-x.y.z.exe
npm run release      # both of the above
```

**The Android signing key.** `android/keystore.properties` (not in git) points at `D:/GitHub/Aurora-keys/aurora-release.jks`. Every APK you ship must be signed with this same key; otherwise Android will not update an installed Aurora and the user would have to uninstall first (losing the app's data). **Back up the Aurora-keys folder.**

**Publishing** is described step by step in `docs/RELEASING.md`.

## Privacy and security model

- **Open source (MIT).** All code is public; there is no hidden server component.
- **No accounts, no analytics, no ads.** There is no tracking code anywhere in `web/`, `android/` or `electron/`.
- **Read-only access to your songs.** Android: a user-granted, revocable folder permission (Storage Access Framework). Windows: the folder path you pick, read through a local `app://` handler that only serves audio/lyrics files. The web app: a browser folder handle with read permission. Songs are never uploaded or copied (except in the fallback for browsers that cannot link a folder, where the browser keeps a private copy).
- **Network use is limited to three things**, listed on the website: update checks (the GitHub Pages site), album cover lookups (Apple's public search API; artist/album names only; can be disabled), and the optional AI DJ lines (Anthropic API, only if the user adds their own key, which is stored only on the device). The Android bridge only allows `itunes.apple.com` and `*.mzstatic.com` for cover lookups.
- **Signed releases and checksums.** The APK is signed with the Aurora release key; SHA-256 checksums of each release are published on the Downloads page. The Windows installer is not yet code-signed (SmartScreen may warn).
- **Not audited.** No third-party security audit has been done; the open code is the basis for trust.

## The website (what each page is)

| URL | Source | Purpose |
|---|---|---|
| `/` | `tools/site/index.html` | Explains the features, the AI DJ, the platforms, and has the **Open source and security** section (what Aurora connects to, how to verify downloads) |
| `/downloads/` | `tools/site/downloads/index.html` | Latest Android and Windows downloads with size and SHA-256, install steps, and every release with its notes. Reads `/releases.json` |

| URL | Source | Purpose |
|---|---|---|
| `/app/` | `web/` | The web app itself, plus `update-manifest.json` (the over-the-air update feed) |
| `/releases.json` | generated | Changelog + download links + checksums, built from `docs/changelog.json` and `downloads/hashes.json` |

`npm run site` builds all of this into `./site` (not committed); GitHub Actions does the same on every push and publishes it.

## 11. Troubleshooting

| Symptom | Cause / fix |
|---|---|
| No sound while testing with Windows Phone Link | Phone Link can send the phone's audio to the PC. Turn that off, or check the PC volume |
| No notification player on Android | Allow notifications for Aurora (Settings inside the app, or Android settings), then play a song |
| "Add music" asks for a folder but the library stays empty | The folder has no supported audio files, or access was refused. Pick it again and tap *Use this folder*, then *Allow* |
| Songs missing after reinstalling Android app | Uninstalling erases the app's data (Android rule). Install new versions *over* the old one, never uninstall first; after a fresh install, pick the folder again (no copying, it takes seconds) |
| Covers missing | The files have no picture and no match was found online. Add a `cover.jpg` to the folder, or edit the song's info and choose a cover |
| Update prompt never appears | Pages not enabled, or the build number was not increased (run `npm run bump`), or the device is offline |
| `Gradle ... Unable to establish loopback connection` | Run the Gradle build from a normal user terminal (not a sandboxed shell) |
| Windows installer fails at the 7-Zip step | Use `compression: normal` (already configured); `maximum` crashes on some machines |

## 12. Glossary

- **APK**: the Android app file. **PWA**: a website that installs like an app. **OTA**: over-the-air update.
- **Capacitor**: tool that wraps a web app into an Android app. **Electron**: same for Windows/desktop.
- **Service worker**: a background script of a web app that caches files for offline use.

- **SAF (Storage Access Framework)**: Android's safe "pick a folder and grant access" system.
- **Range request**: asking a server/app for just part of a file, which makes seeking in a song instant.
- **Ducking**: lowering the music volume while the DJ speaks.
- **ID3 / MP4 tags**: song information stored inside audio files (title, artist, cover...).

PART 2

# Releasing Aurora and publishing updates

This is the exact, step-by-step process. Your GitHub user is `Kailu687` and the repository is called `Aurora`, so:

| What | Address |
|---|---|
| Website | `https://kailashamarnath.me/Aurora/` |
| Downloads page | `https://kailashamarnath.me/Aurora/downloads/` |
| Web app | `https://kailashamarnath.me/Aurora/app/` |
| Update feed the apps read | `https://kailashamarnath.me/Aurora/app/update-manifest.json` |
| Source code | `https://github.com/Kailu687/Aurora` |

Because your GitHub account already publishes `kailashamarnath.me` (custom domain), every project site under the account is served from it, so Aurora lives at `kailashamarnath.me/Aurora/`. The old `kailu687.github.io/Aurora/` address redirects there. The apps know both: `web/config.json` has `updateUrl` (the custom domain) and `updateUrlFallback` (the `github.io` address), and they try them in that order. If you ever rename the repository or user, change it there, in `tools/site/**`, in `tools/publish-web.js` (the `REPO` constant) and in `package.json`.

## A. One-time setup (about 10 minutes)

1. **Publish the repository** with GitHub Desktop. File, Add local repository, choose `D:\GitHub\Aurora` (it is already a git repository with the first commit made). Press **Publish repository**, name it `Aurora`, keep it **Public** (GitHub Pages is free for public repositories), and make sure the branch is `main`.
2. **Turn on GitHub Pages.** On github.com open the repository, **Settings, Pages**, and under *Build and deployment, Source* choose **GitHub Actions**.
3. **First deploy.** Open the **Actions** tab. The workflow "Publish website and updates" runs on every push to `main` (or press *Run workflow*). When it shows a green tick the site is live.
4. **Create the first GitHub Release** (this is where the Windows installer lives, section C).
5. **Back up the signing key folder** `D:\GitHub\Aurora-keys\`. It is deliberately *not* in the repository.

## B. Publishing a new version (every time)

1. Change whatever you want under `web/` (or have it changed for you).
2. Bump the version. Separate release-note bullets with `|`; they appear on the Downloads page:

```
npm run bump -- --notes "New equalizer presets|Fixed a crash when seeking"
```

This updates the version everywhere (web, Android, Windows) and adds the entry to `docs/changelog.json`.

3. Build the files (only needed when you want users to get a *new APK or installer*, see section D):

```
npm run release        # APK + Windows installer + checksums
```

or separately `npm run android` / `npm run windows` and then `npm run hash`.

4. In GitHub Desktop: write a summary, **Commit to main**, then **Push origin**.
5. GitHub Actions rebuilds the website in about a minute. Every installed Aurora then notices the new build number, shows "*Update available*", and after the user taps **Update now** downloads it and restarts.

## C. Attaching the files to a GitHub Release (so the Windows button works)

The Windows installer (about 87 MB) is too large for git, so it is published as a GitHub **Release**:

1. github.com, the repository, **Releases, Draft a new release**.
2. Tag: `v<version>` (for example `v1.6.0`). Title: `Aurora <version>`. Paste the release notes.
3. Attach these files (installer from `dist\`, APK from `downloads\`):
   - `Aurora-Setup-<version>.exe`
   - `Aurora.apk` (the file in `downloads\`; use exactly this name) Keep these exact names; the Downloads page builds its per-release links from them.
4. **Publish release.**

The Downloads page lists the latest APK straight from the website ( `/downloads/Aurora.apk`, which is committed to git), and the Windows installer plus all older versions from the Releases.

## D. Which changes reach users how

| You changed | Users get it by |
|---|---|
| Anything under `web/` (screens, features, DJ, styles, fixes) | **Over-the-air update**, no reinstall |
| `android/` Java code, permissions, icons | A **new APK**: `npm run android`, commit `downloads/Aurora.apk`, publish; the user installs it *over* the old one |
| `electron/main.js`, `electron/preload.js` | A **new Windows installer** |

A new APK or installer also contains the latest `web/`, so fresh installs are always current.

## E. Checking that updates work

1. Install the current release on a phone or PC.
2. Make a small visible change in `web/`, run `npm run bump -- --notes "test"`, commit, push, wait for the green tick.
3. Open Aurora. Within seconds you should get *Update available*. (Or Settings, Cloud updates, **Check for updates**.)

## F. Troubleshooting releases

- **Site shows 404**: Pages is not set to *GitHub Actions* (step A2), or the first workflow run has not finished.
- **Apps say "up to date" after a push**: the build number did not change. Run `npm run bump` and push again.
- **Windows button says "not found"**: the GitHub Release (section C) has not been published yet.
- **"App not installed" when updating the APK**: the new APK was signed with a different key. Always build with `npm run android` on this PC (it uses `D:\GitHub\Aurora-keys`). An installed copy signed with another key must be uninstalled once (its saved data is lost).
- **Actions fails at "deploy"**: check Settings, Pages, Source = GitHub Actions, and that the repository is public.
- **Gradle says "Unable to establish loopback connection"**: run `npm run android` from a normal terminal.